Table 3: Zero-shot ranking performance comparison. We highlight the best performance in **bold**. Due to limited budget, we evaluate each LLM only once on 200 sampled users **only** for experiments corresponding to this table.

Method	ML-1M				Games			
	N@1	N@5	N@10	N@20	N@1	N@5	N@10	N@20
BM25 [50]	4.70	12.68	17.88	33.19	13.92	28.81	34.61	44.35
UniSRec [30]	7.37	18.80	26.67	37.93	18.95	33.99	40.71	48.42
Alpaca-7B [55]	4.00	13.92	23.09	31.54	5.50	14.16	21.67	28.68
Vicuna-13B [9]	6.50	14.75	22.64	33.42	7.00	17.73	24.30	31.22
LLaMA-2-70B-Chat [57]	8.00	25.42	31.19	34.52	21.50	32.30	37.83	41.97
ChatGPT (GPT-3.5)	**23.33**	**42.07**	**48.80**	**53.73**	23.83	45.69	50.31	55.45
GPT-4	15.50	40.65	46.74	48.42	**39.50**	**58.22**	**62.88**	**65.25**

measured by item frequency in one recommendation dataset. In Figure 3(c), we report the item popularity score (measured by the normalized item frequency of appearance in the training set) at each position of the ranked item lists. We can see that popular items tend to be ranked at higher positions.

Making LLMs focus on historical interactions helps reduce popularity bias. We assume that if LLMs focus on historical interactions, they may give more personalized recommendations but not more popular ones. From Figure 2(b), we know that LLMs make better use of historical interactions when using less historical interactions. From Figure 3(d), we compare the popularity scores of the best-ranked items varying the number of historical interactions. It can be observed that as $|\mathcal{H}|$ decreases, the popularity score decreases as well. This suggests that one can reduce the effects of popularity bias when LLMs focus more on historical interactions. From the above experiments, we can conclude the following:

Observation 2. LLMs suffer from position bias and popularity bias while ranking, which can be alleviated by bootstrapping or specially designed prompting strategies.

3.3 How Well Can LLMs Rank Candidates in a Zero-Shot Setting?

We further evaluate LLM-based methods on candidates with hard negatives that are retrieved by different strategies to further investigate what the ranking of LLMs depends on. Then, we present the ranking performance of different methods on candidates retrieved by multiple candidate generation models to simulate a more practical and difficult setting.

LLMs have promising zero-shot ranking abilities. In Table 2, we conduct experiments to compare the ranking abilities of LLM-based methods with existing methods. We follow the same setting in Section 3.1 where $|\mathcal{C}| = 20$ and candidate

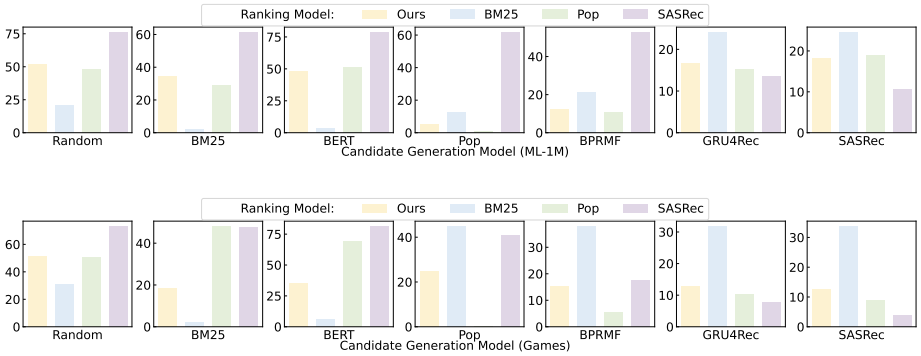

Fig. 4: Ranking performance measured by NDCG@10 (%) on hard negatives.

items are randomly retrieved. We include three conventional models that are trained on the training set, *i.e.,* Pop (recommending according to item popularity), BPRMF [49], and SASRec [33]. We also evaluate three zero-shot recommendation methods that are not trained on the target datasets, including BM25 [50] (rank according to the textual similarity between candidates and historical interactions), UniSRec [30], and VQ-Rec [29]. For UniSRec and VQ-Rec, we use their publicly available pre-trained models. We do not include ZESRec [15] because there is no pre-trained model released. In addition, we compare the zero-shot ranking performance of different LLMs in Table 3. "Recency-Focused" prompting strategy is used for LLM-based rankers.

From Table 2 and 3, we can see that LLMs with more parameters generally perform better. The best LLM-based methods outperform existing zero-shot recommendation methods by a large margin, showing promising zero-shot ranking abilities. We would highlight that it is difficult to conduct zero-shot recommendations on the ML-1M dataset, due to the difficulty in measuring the similarity between movies merely by the similarity of their titles. However, LLMs can use their intrinsic knowledge to measure the similarity between movies and make recommendations. We would emphasize that the goal of evaluating zero-shot recommendation methods is not to surpass conventional models. The goal is to demonstrate the strong recommendation capabilities of pre-trained base models, which can be further adapted and transferred to downstream scenarios.

LLMs rank candidates based on item popularity, text features as well as user behaviors. To further investigate how LLMs rank the given candidates, we evaluate LLMs on candidates that are retrieved by different candidate generation methods. These candidates can be viewed as hard negatives for ground-truth items, which can be used to measure the ranking ability of LLMs for specific categories of items. We consider two categories of strategies to retrieve the candidates: (1) *content-based methods* like *BM25* [50] and *BERT* [14] retrieve candidates based on the text feature similarities, and (2) *interaction-*

Table 4: Performance comparison on *candidates retrieved by multiple candidate generation models*. Ground-truth items are *not* guaranteed to be included in the candidate sets. "full" denotes models that are trained on the target dataset, and "zero-shot" denotes models that are not trained on the target dataset but could be pre-trained. We highlight the best and second-best performance among *all* recommendation methods in **bold**.

	Method	ML-1M				Games			
		N@1	N@5	N@10	N@20	N@1	N@5	N@10	N@20
full	Pop	0.08	1.20	4.13	5.79	0.13	1.00	2.27	2.62
	BPRMF [49]	0.26	1.69	4.41	6.04	0.55	1.98	**2.96**	**3.19**
	SASRec [33]	**3.76**	**9.79**	**10.45**	**10.56**	**1.33**	**3.55**	**4.02**	**4.11**
zero-shot	BM25 [50]	0.26	0.87	2.32	5.28	0.18	1.07	1.80	2.55
	UniSRec [30]	0.88	3.46	5.30	6.92	0.00	1.86	2.03	2.31
	VQ-Rec [29]	0.20	1.60	3.29	5.73	0.20	1.21	1.91	2.64
	Ours	**1.74**	**5.22**	**6.91**	**7.90**	**0.90**	**2.26**	2.80	3.08

based methods, including *Pop*, *BPRMF* [49], *GRU4Rec* [28], and *SASRec* [33], retrieve items using neural networks trained on user-item interactions. Given candidates, we compare the ranking performance of the LLM-based model (*Ours*) and representative methods.

From Figure 4, we can see that the ranking performance of the LLM-based method varies on different candidate sets and different datasets. (1) On ML-1M, LLM-based method cannot rank well on candidate sets that contain popular items (*e.g., Pop* and *BPRMF*), indicating the LLM-based method recommend items largely depend on item popularity on ML-1M dataset. (2) On Games, we can observe that *Ours* has similar performance both on popular candidates and textual similar candidates, showing that item popularity and text features contribute similarly to the ranking of LLMs. (3) On both two datasets, the performance of *Ours* is affected by hard negatives retrieved by interaction-based candidate generation models, but not as severe as those interaction-based rankers like *SASRec*. The above results demonstrate that LLM-based methods not only consider one single aspect for ranking, but make use of item popularity, text features, and even user behaviors. On different datasets, the weights of these three aspects to affect the ranking performance may also vary.

LLMs can effectively rank candidates retrieved by multiple candidate generation models. For real-world recommender systems [10], the items to be ranked are usually retrieved by multiple candidate generation models. As a result, we also conduct experiments in a more practical and difficult setting. We use the above-mentioned seven candidate generation models to retrieve items. The top-3 best items retrieved by each candidate generation model will be merged into a candidate set containing a total of 21 items. As a more practical setting, we do not complement the ground-truth item to each candidate set. Note that

the experiments here were conducted under the implicit preference setup [77], indicating that implicit positive instances (not explicitly labeled) may exist among the retrieved items. A more faithful evaluation might require a human study, which we intend to explore in our future work. For *Ours*, we summarize the experiences gained from Section 3.1 and 3.2. We use the recency-focused prompting strategy to encode $|\mathcal{H}| = 5$ sequential historical interactions into prompts and use a bootstrapping strategy to repeatedly rank for 3 rounds.

From Table 4, we can see that the LLM-based model (*Ours*) yields the second-best performance over the compared recommendation models on most metrics. The results show that LLM-based zero-shot ranker even beats the conventional recommendation model *Pop* and *BPRMF* that has been trained on the target datasets, further demonstrating the strong zero-shot ranking ability of LLMs. We assume that LLMs can make use of their intrinsic world knowledge to rank the candidates comprehensively considering popularity, text features, and user behaviors. In comparison, existing models (as *narrrow experts*) may lack the ability to rank items in a complicated setting. The above findings can be summarized as:

Observation 3. LLMs have promising zero-shot ranking abilities, especially on candidates retrieved by multiple candidate generation models with different practical strategies.

4 Related Work

Transfer learning for recommender systems. As recommender systems are mostly trained on data collected from a single source, people have sought to transfer knowledge from other domains [71,85,45,86,76,83], markets [3,51], or platforms [4,19]. Typical transfer learning methods for recommender systems rely on anchors, including shared users/items [45,84,69,70,7,8] or representations from a shared space [11,18,38]. However, these anchors are usually sparse among different scenarios, making transferring difficult for recommendations [85]. More recently, there are studies aiming to transfer knowledge stored in language models by adapting them to recommendation tasks via tuning [1,21,12,53] or prompting [37,75,39]. In this paper, we conduct zero-shot recommendation experiments to examine the potential to transfer knowledge from LLMs.

Large language models for recommender systems. The design of recommendation models, especially sequential recommendation models, has been long inspired by the design of language models, from word2vec [2,22,25] to recent neural networks [28,33,82,54]. In recent years, with the development of pre-trained language models (PLMs) [14], people have tried to transfer knowledge stored in PLMs to recommendation models, by either representing items using their text features or representing behavior sequences in the format of natural language [21,58,42,16,68]. Very recently, large language models (LLMs) have been shown superior language understanding and generation